When Learned IDS Model Selection Collapses to a Static Choice: A Measurement Study at the Edge

Nuonan Ouyang, Adrian Shatte, Zhigang Lu, Chao Chen, *Member, IEEE,* and Wei Xiang, *Senior Member, IEEE*

***Abstract*—Runtime model selection trades detection quality against energy and latency, but comparison only with a costlier static detector conflates detector choice with adaptive value. We present an analytic and physical measurement framework combining a pairwise reward criterion, matched static controls, paired blocks, a frozen load–prevalence factorial, and off-support action reporting. On one Raspberry Pi 4B with four frozen TON-IoT detectors, tabular Q-learning and DQN realize a static LightLR schedule. A retrospective validation-only audit excludes MedRF/LightLR and HeavyMLP/LightLR preference reversals but identifies seven TinyDT-favouring windows among 2,500; it does not prove global LightLR optimality. Under a $50/100/4000$ rows/s$\times 0.10/0.50/0.90$ factorial, a causal DeadlineGuard saves 43.7 J per 500-s run against Static-MedRF (2.98%; 95% paired CI 41.4–46.0 J) and eliminates 512/5,000 deadline misses at a 1.373-percentage-point F1 cost. Off training support, Tabular-Q's deterministic tie rule selects TinyDT and reaches 67.76% false-positive rate. These single-board results motivate joint auditing of reward attainability, realized actions, matched-static attribution, service feasibility, and off-support defaults.**

***Index Terms*—Intrusion detection, IoT, edge resource management, model selection, reinforcement learning, service management, energy measurement, reproducibility.**

I. Introduction

Edge intrusion detection must trade security against latency and energy on tightly constrained hardware [1], [2], [3]. Runtime model selection appears attractive, but a learned controller may simply choose one cheap detector. Comparing it only with a costlier static detector then attributes an operating-point change to adaptation. Reward normalization can create the same problem before training by making action reversal unattainable, while sparse discrete states can hide an unsafe default action until the workload moves off support.

(*Corresponding author: Nuonan Ouyang.*)

N. Ouyang and A. Shatte are with the College of Science and Engineering, Division of Research, James Cook University, Bebegu Yumba Campus, Building 17, Douglas, Townsville, QLD, Australia (e-mail: nuonan.ouyang@my.jcu.edu.au; adrian.shatte@jcu.edu.au; ORCID: 0009-0004-7124-8940; 0000-0002-6225-9697).

Z. Lu is with the School of Computer, Data and Mathematical Sciences, Western Sydney University, Penrith (Kingswood) Campus, Kingswood, NSW, Australia (e-mail: z.lu@westernsydney.edu.au; ORCID: 0000-0001-5102-6217).

C. Chen is with the College of Business and Law, RMIT University, Melbourne, VIC 3000, Australia (e-mail: chao.chen@rmit.edu.au; ORCID: 0000-0003-1355-3870).

W. Xiang is with the Australian Centre for AI in Medical Innovation and the Cisco-La Trobe Centre for AI and Internet of Things, La Trobe University, Melbourne, VIC, Australia (e-mail: w.xiang@latrobe.edu.au; ORCID: 0000-0002-0608-065X).

We address these risks with an analytic and measurement framework. First, a pairwise non-degeneracy criterion decides from the declared reward, cost profile, and validation support whether two actions can ever exchange rank. Second, a five-instrument protocol requires trace-matched physical blocks, the static action actually realized by the selector, a prospectively frozen factorial, joint raw/drift-adjusted endpoints, and full on-/off-support action reporting. Third, we apply the framework on one Raspberry Pi 4B with four frozen TON-IoT detectors, Tabular-Q, DQN, a fixed confidence cascade, and a causal DeadlineGuard rule. Under the frozen $50/100/4000$ rows/s $\times$ $0.10/0.50/0.90$ factorial, DeadlineGuard saves 43.7 J per 500-s run against Static-MedRF (2.98%; 95% paired CI 41.4–46.0 J) and eliminates all 512/5,000 deadline misses at a 1.373-pp F1 cost. The learned selectors instead realize LightLR on observed support; off support, Tabular-Q's deterministic tie rule selects TinyDT and yields 67.76% FPR.

Four questions organize the study: **RQ1**, what security/resource operating points anchor the frozen detector pool? **RQ2**, do learned selectors beat the static schedule they realize, and does their reward admit preference reversal? **RQ3**, do confidence- and deadline-aware controls switch usefully under fixed and variable load? **RQ4**, how do state coverage, deadline violations, and validation-only threshold calibration constrain deployment claims? Our contribution is not a new RL algorithm; it is a reproducible service-management method that separates detector choice, conditional adaptation, objective design, and off-support safety. Figure 1 summarizes the physical apparatus and evidence path.

II. Related Work

Our work sits at the intersection of edge IDS, conditional inference, model serving, and learned resource orchestration. Recent IDS research covers continual detection, heterogeneous representations, and runtime model selection [4], [5], [6], [7]. Auto-IDaS and FEDMS show that learned or confidence-driven detector selection is operationally plausible [8], [9], but neither a complex controller nor a cheaper operating point by itself establishes adaptive value. We therefore require comparison with the static detector actually realized by the selector and expose the full state–action map, not only visited-state occupancy.

Conditional-computation systems—including BranchyNet, adaptive network selection, MSDNet, and shallow–deep

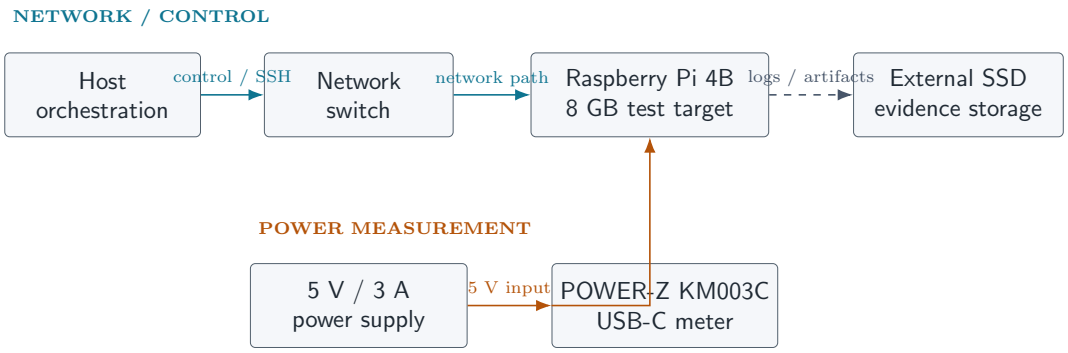


Fig. 1. Schematic apparatus overview. A 5-V supply feeds the Raspberry Pi 4B through the POWER-Z KM003C USB-C meter; network orchestration reaches the Pi via the switch; the external SSD stores evidence artifacts. The drawing shows roles and paths only.

networks—escalate uncertain inputs to costlier models [10], [11], [12], [13]. Our fixed confidence cascade translates that idea to whole-device IDS measurement. It also exposes a serving detail often hidden by row-level escalation rates: only 1.34% of rows escalate, yet they activate MedRF in 66.76% of windows, so call granularity rather than row fraction governs the observed cost.

Serving systems such as Clipper, INFaaS, and Clockwork make accuracy, latency, and cost SLOs explicit and route requests among model variants [14], [15], [16]. We adopt their service-contract perspective but add two missing controls: a matched static action that separates routing value from model choice, and a pre-deployment audit that asks whether the declared reward permits any action reversal on validation support.

RL-based edge orchestration spans vRAN, vehicle–edge scheduling, DVFS, offloading, and QoS control [2], [1], [3], [17], [18], [19], [20], [21], [22]. Tabular Q-learning and DQN are representative discrete and neural value-based controllers [23], [24]. Our contribution is not a new RL algorithm. It is a diagnostic methodology that combines an analytic reward screen, matched physical attribution, a prospectively frozen load/prevalence factorial with an explicit deadline SLO, and off-support action auditing. These instruments address known evaluation fragility in security ML and small-sample RL studies [25], [26].

III. Compact System Model and Service Objectives

An edge node processes batches in decision windows $t = 1, 2, \ldots$ using a resident detector pool $\mathcal{M} = \{m_1, \ldots, m_K\}$. Before inference, a frozen controller commits one action $a_t = \pi(s_t) \in \mathcal{M}$ from the causal state

$$s_t = (\widetilde{T}_t, h_{t-1}, q_t, \lambda_{t-1}, a_{t-1}), \tag{1}$$

where $\widetilde{T}_t$ is smoothed temperature, h_{t-1} the preceding selected detector's mean posterior, q_t queue utilization, λ_{t-1} preceding arrival count, and a_{t-1} preceding action. Ground truth, current outputs of unselected detectors, and factorial-cell identity are unavailable at decision time. Because all models remain resident, selection excludes fitting and model-loading cost.

For each completed window we record attack-class F1, recall, and false-positive rate; completion latency is checked against deadline Δ_{SLO}; and resource cost is whole-device USB energy. In the physical study $\Delta_{\mathrm{SLO}} = 1$ s and each run lasts 500 s. Training optimizes a scalar surrogate, but evaluation jointly reports security, timeliness, energy, action occupancy, and state support. Test policies are frozen and never updated online.

The network-layer adversary may vary traffic prevalence and arrival rate but cannot alter the meter, controller memory, detector artifacts, or training pipeline. Adversarial-ML attacks and hardware tampering are outside scope. The operational risk considered here is induced entry into unvisited discrete states: deterministic ties can then expose an unaudited default action.

We credit a selector with *adaptive value* only when it realizes more than one action on deployment support, improves a service objective without a comparable regression, and survives a paired comparison with the static instance of its dominant realized action. Otherwise the measured difference is attributed to detector operating-point choice rather than conditional adaptation.

IV. Dataset-Native Data Preparation

A. Choice of Primary Dataset

The primary systems study uses TON-IoT [27]. Its records combine IoT service telemetry, operating-system events, and networked interactions labeled with attack categories, making the dataset representative of the mixed-signal traffic that reaches an edge IDS at the service boundary rather than at the wire only. We deliberately restrict the physical dynamic-power study to a single dataset because coupling multiple encoded pipelines to one hardware campaign would confound feature dimensionality, class balance, and physical measurement; instead CICIoT2023 [28] and N-BaIoT [29] are retained as supplementary detector and coverage audits and are not used to imply cross-dataset dynamic-power generalization.

B. Frozen Preprocessing and Splits

The TON-IoT pipeline is frozen and retains 32 native features. Imputation, scaling, and categorical encoding are fitted on training data only and applied deterministically to validation and test. The split groups unordered IP pairs so

that traffic patterns are not divided across partitions; the grouping rule hashes "11|" concatenated with the sorted IP pair and thresholds the first sixteen SHA-256 hexadecimal digits modulo 100 (training below 60, validation 60–79, test 80–99). Training seed 11 fixes all model fits, so any single-seed conclusion is explicit rather than implied. Source-row identifiers and raw-field-string fingerprints are cross-partition disjoint, and an independent encoded-float32 audit finds *zero* TON-IoT train–test vector matches, which addresses a commonly-cited leakage risk in IoT IDS benchmarks [25].

C. Auxiliary Dataset Audits

The same ledger retains dataset-native CICIoT2023 and N-BaIoT pipelines with 39 and 115 native features respectively. For CICIoT2023, seventeen attack labels remain training-only because their capture files are too few for a defensible test split; the actual test taxonomy is a binary attack-versus-normal partition over sixteen attack categories plus benign traffic. For N-BaIoT, physical device identifiers are the grouping unit, giving five training, two validation, and two test devices; a post-hoc byte-level audit records that 17.3% of N-BaIoT test rows share exact float32 vectors with training rows (seven distinct vectors), reflecting the sensor-repetition structure of the dataset rather than a preprocessing fault. These coverage and encoded-overlap limitations are retained rather than used to imply cross-dataset dynamic-power generality; consequently the auxiliary datasets support only static classification and replay evidence.

D. Class Balance and Replay Traces

The TON-IoT test set contains a majority attack fraction, which is common in IoT benchmarks and which we do not rebalance because the physical study measures whole-device behavior on the natural distribution. To probe operating points under controlled prevalence shifts, the supplement retains twenty static replay traces per model (fifty per dataset before deduplication), with successive phase attack fractions $\{0.05, 0.35, 0.70, 0.40, 0.10\}$ over $\{100, 100, 120, 100, 80\}$ windows. These traces do not constitute a new detector-training exercise; they resample fixed held-out pools to expose how a frozen detector's F1 and balanced accuracy behave when prevalence, not distribution, changes.

V. Detector Pool, Reward Specification, and Non-degeneracy Framework

This section specifies the frozen detector artifacts, the reward specification used by both learned controllers, and the pre-deployment pairwise non-degeneracy screening test. The last of these is stated as a theorem so that it can be applied to future learned selectors without re-derivation.

A. Detector Pool

Each dataset has four separately fitted configurations: depth-6 TinyDT; L2-regularized LightLR; 25-tree, depth-10 MedRF; and a 64/32/16 HeavyMLP. All use threshold 0.5 and training seed 11. Detector names do not assert a resource ranking; the physical measurements determine the empirical order. During TON-IoT physical experiments all models remain resident, and an action selects one model for the next batch without loading, fitting, or updating it. This design deliberately isolates the effect of controller decisions from model instantiation cost so that any observed switching cost is attributable to inference itself, not to hot-swapping detectors.

B. Causal State Encoding

Before window t the controller constructs s_t per (1). The kernel temperature is smoothed as $\widetilde{T}_t = 0.9\widetilde{T}_{t-1} + 0.1T_t$, initialized at T_0. The threat observation h_{t-1} is the arithmetic mean attack probability from the single detector selected in the preceding window; it is *not* the preceding ground-truth attack fraction. Queue utilization is bounded to $[0, 1]$, and the preceding arrival count is divided by 100 and bounded to $[0, 2]$. Initial values are $h_0 = 0.5$, $\lambda_0/100 = 1$, and $a_0 = \text{TinyDT}$.

Each of the first four state variables has three bins. Their two boundaries are $(45, 70)$ °C, $(1/3, 0.7)$ for threat, $(1/3, 2/3)$ for queue utilization, and $(0.9, 1.1)$ for arrival ratio. Including four previous actions yields $3^4 \times 4 = 324$ encoded states. Boundary equality enters the higher bin. The flat index is row-major in the listed order, with previous action changing fastest.

The executed order within a window is: (i) state construction from causal telemetry and lagged context; (ii) action commitment and log; (iii) selected-detector inference; (iv) outcome log; and (v) lagged-state update for $t + 1$. Ground truth, cell identity, and unselected current-window outputs are unavailable online. This information boundary is what enables preference-reversal analysis to be conducted purely on state features that the deployment can observe.

C. Reward and Cost Provenance

The unchanged per-window training reward is

$$r_t = 0.5\, B_t \;-\; 0.2\, \overline{P}_t \;-\; 0.2\, \overline{L}_t \;-\; 0.1\, \overline{H}_t, \tag{2}$$

where B_t is balanced accuracy over the classes present in the window. With $[x]_0^1 = \min(1, \max(0, x))$, the normalization is

$$\overline{P}_t = \left[\tfrac{P_t - 2.347219336}{0.049806200}\right]_0^1, \qquad \overline{L}_t = \left[\tfrac{L_t - 1.2979595}{29.6547998}\right]_0^1, \tag{3}$$

$$\overline{H}_t = \left[\tfrac{\max(0, T_t^{\text{post}} - \widetilde{T}_t)}{1\,^\circ\text{C}}\right]_0^1. \tag{4}$$

P_t and L_t are frozen TON-IoT paced-profile lookups, not contemporaneous training measurements. Their means, listed as (power W, batch latency ms) for the four detectors, are 2.353132/1.297960 for TinyDT, 2.347219/1.544420 for LightLR, 2.397026/30.952759 for MedRF, and 2.366688/4.417836 for HeavyMLP. The compact software traces contain no post-action temperature response: their adapter supplies a fixed 35°C predecision value and reuses that action-invariant value in the outcome field. Thus $\overline{H}_t = 0$ throughout the executed training and validation traces; it is not an action-level thermal cost. No fitted thermal response or switching-energy penalty is used, and consequently no thermal control action is claimed.

D. Pairwise Non-degeneracy Criterion

Reward normalization is often reported as a hyperparameter without evidence that it permits any state to reverse the preferred action. We now show that this question can be answered before deployment with only the declared weights, cost profile, and a support region.

Consider two actions $a, b \in \mathcal{M}$. Decompose the scalar utility into a detection contribution and a normalized resource cost:

$$U_a(s) = w_D\, D_a(s) - C_a, \qquad C_a = \sum_j w_j\, \overline{C}_{j,a}, \quad (5)$$

where $w_D = 0.5$ in our specification and $\overline{C}_{j,a}$ collects the normalized resource penalties that are fixed on the declared support (as they are for the executed paced profiles at neutral thermal cost). Let $\Delta D_{ab}(s) = D_a(s) - D_b(s)$ and $\Delta C_{ab} = C_a - C_b$. Then

$$U_a(s) - U_b(s) = w_D\, \Delta D_{ab}(s) - \Delta C_{ab}, \qquad \tau_{ab} = \frac{\Delta C_{ab}}{w_D}. \quad (6)$$

Theorem 1 (Pairwise non-degeneracy)**.** *Let $\mathcal{S}$ be a support region. Under the frozen scalar utility* (6)*, a strict pairwise preference reversal between actions a and b exists on $\mathcal{S}$ if and only if*

$$\min_{s\in\mathcal{S}} \Delta D_{ab}(s) < \tau_{ab} < \max_{s\in\mathcal{S}} \Delta D_{ab}(s). \quad (7)$$

If (7) *fails, every $s \in \mathcal{S}$ yields the same sign of $U_a(s) - U_b(s)$; consequently no policy that maximizes the frozen utility can prefer a on one state and b on another within $\mathcal{S}$.*

Corollary 2 (Attainability under bounded detection)**.** *If $D_a(s), D_b(s) \in [0,1]$ for every $s \in \mathcal{S}$, then reversal is impossible whenever $|\tau_{ab}| \geq 1$; and reversal is guaranteed impossible whenever the attainable interval $[\min \Delta D_{ab}, \max \Delta D_{ab}]$ does not straddle τ_{ab}.*

Because balanced accuracy lies in $[0,1]$, the corollary gives an immediate ceiling. Under the executed candidate-range scaling, the frozen nonthermal penalties imply $\tau_{\text{MedRF,LightLR}} = 0.7967$. A reproducible audit of all 2,500 windows in the five hash-bound validation traces finds $\Delta D_{\text{MedRF,LightLR}} \in [-0.1053, 0.1053]$, so this threshold lies outside the observed validation-window support. This retrospective audit excludes MedRF/LightLR and HeavyMLP/LightLR reversal on observed validation support. TinyDT/LightLR can reverse: TinyDT has higher immediate reward in 7 of 2,500 validation windows. These label-informed window comparisons establish neither optimality over all encoded states nor optimal long-horizon return, and do not fully explain the learned collapse. There is no evidence that this audit preceded test inspection. Supplementary Theorem S1 gives the proof, Supplementary Table S11A gives the full pairwise matrix, and Supplementary Algorithm S1 gives the validation-only reward audit. If reversal is unattainable, the engineer should use an audited static schedule or renormalize against an external deployment contract—for example idle-relative power, the latency SLO, an energy envelope, or thermal headroom—rather than against detector-pool extrema.

E. Policies

Four policies are evaluated in the physical study. *Static-{TinyDT, LightLR, MedRF, HeavyMLP}* freeze a single detector. *CFSM* is a fixed heuristic initialized to TinyDT; at a smoothed temperature of at least 70 °C it steps down one model rank, bypassing cooldown; otherwise it holds for two windows after a switch and steps down/up one rank when threat crosses 0.7. Queue and load are logged but not used by CFSM. *Tabular-Q* uses zero initialization, $\alpha = 0.1$, $\gamma = 0.9$, and the update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + 0.1\, \delta_t, \qquad \delta_t = r_t + 0.9 \max_a Q(s_{t+1}, a) - Q(s_t, a_t), \quad (8)$$

with zero terminal bootstrap [23]. *DQN* uses one-hot state, two 64-unit ReLU layers, Adam 10^{-3}, replay 10,000, batch 32, and a target copy every 250 updates [24]. Both learners train for 1,000 episodes with ε linearly annealed from 1 to 0.01; checkpoints are selected every 25 episodes on five validation traces. Test execution freezes the policy and performs no update.

Two prospectively frozen causal policies join the reviewer campaigns. *ConfidenceCascade* first executes LightLR on the full 100-row batch and, when its posterior p_{LR} falls in $[0.3, 0.7]$, invokes MedRF once on the deferred sub-batch. *Deadline-Guard* uses only the current causal arrival count N: it selects MedRF when the predicted MedRF cost $\lceil N/100 \rceil L_{95}^{\text{RF}} \leq 1$ s, otherwise LightLR. Both thresholds were frozen from pre-experiment latency data.

Archived process-start and policy-creation timestamps indicate approximately 26.7 s for the Tabular-Q training run and 256.9 s for DQN; these are provenance-derived wall-clock intervals rather than calibrated compute benchmarks, and training energy was not instrumented. Deployment cost is measured separately in Section VIII.

The runtime sequence is causal: observe telemetry and lagged context; encode s_t; commit and log a_t; run only the selected resident detector; log outcome and telemetry; then update the lagged state. Ground truth is used only after the outcome during offline training. Formal test deployment freezes the policy and omits the update.

VI. Evaluation Protocol and Measurement Instrumentation

Our evaluation protocol is designed to separate detector choice from adaptive value along five instruments: (i) paired physical blocks across trace-matched detectors; (ii) matched-static-action attribution controls; (iii) prospectively frozen factorial workloads; (iv) separate raw and drift-adjusted power endpoints; and (v) full occupancy and off-support action reporting. This section describes each instrument together with the classification, timing, and power measurement pipelines that feed them.

A. Classification and Static Evidence

The held-out classification evaluation uses the frozen threshold 0.5 and reports attack-class F1, recall, balanced accuracy,

false-positive rate, and the full confusion counts. In particular,

$$\mathrm{FPR} = \frac{FP}{TN + FP}, \qquad F1 = \frac{2TP}{2TP + FP + FN}, \quad (9)$$

$$\mathrm{BA} = \tfrac{1}{2}\left(\tfrac{TP}{TP+FN} + \tfrac{TN}{TN+FP}\right). \quad (10)$$

Counts are pooled over the declared evaluation unit before metrics are calculated. The supplement retains 50 static traces and 200 model–trace evaluations; these reuse fixed held-out pools and do not constitute fresh attacks, classifier retraining, or physical repeats.

B. Pi Prediction Consistency and Inference Timing

Pi parity checks cover 12 dataset–model combinations and 10,000 encoded rows each with *zero* threshold-label disagreements against the training-host outputs. Tight-loop timing reports 100-row inference after warm-up and excludes encoding, transport, queueing, and controller cost; these boundaries are re-stated wherever a timing figure is used.

C. Static Whole-Device Power

The physical reference uses one Raspberry Pi 4B 8 GB and an inline POWER-Z KM003C USB-C meter. Fifteen valid 1,800-second segments comprise three repeats of instrumented idle and four TON-IoT detectors, all resident, at 100 pre-encoded rows/s. This is resource characterization, not another detection test.

The meter records nominally 1 Hz boundary samples, including raw ADC replies, signed current, voltage, and monotonic request/receive times. Under the recorded connector orientation, consumed power is $P_i = V_i(-I_i)$. Using the observed sampling times τ_i, input energy is approximated by

$$E = \sum_{i=1}^{n-1} \frac{P_i + P_{i+1}}{2}(\tau_{i+1} - \tau_i), \qquad \bar{P} = \frac{E}{\tau_n - \tau_1}. \quad (11)$$

No missing sample is filled. This is 1-Hz load-side USB energy, not calibrated wall-side measurement or a subsecond waveform.

D. Statistical Units and Interval Reporting

For an outcome z observed over n declared repetitions, the two-sided Student-t interval is

$$\bar{z} \pm t_{0.975,\,n-1}\,\frac{s_z}{\sqrt{n}}, \quad (12)$$

using the sample SD [30]. Physical scheduler inference uses *ten paired blocks*, never rows, windows, or meter frames. Raw energy is primary. Drift sensitivity subtracts each run's immediately preceding idle power; it is reported separately rather than substituted for raw measurement. Exact 2^{10} sign-flip tests supply a non-parametric robustness diagnostic under sign-symmetry and exchangeability, and are reported alongside the classical interval when both directional stability and magnitude precision matter.

TABLE I
TON-IoT HELD-OUT DETECTION AT THRESHOLD 0.5. BA DENOTES BALANCED ACCURACY; FPR IS A PERCENTAGE.

Model	F1	Recall	BA	FPR (%)
TinyDT	0.4792	0.9976	0.6599	67.78
LightLR	0.9620	0.9668	0.9766	1.35
MedRF	0.9746	0.9965	0.9906	1.52
HeavyMLP	0.9736	0.9903	0.9883	1.38

E. Physical Campaigns Overview

Six physical campaigns are executed. The *primary* campaign contains ten matched 500-second blocks of Static-MedRF, CFSM, Tabular-Q, and DQN. *P1A* adds ten matched blocks with Static-LightLR to enable the attribution comparison against the static action the learned policies realize. *P1B* isolates state-to-action decision timing. *P1C–P1E* test seed, reward/state, and Tabular-Q α/γ variations. *R1* adds ten matched blocks at 100 rows/s comparing Static-LightLR, the confidence cascade, frozen Tabular-Q, and Static-MedRF. *R2* adds ten matched blocks and six methods over a prospectively frozen $50/100/4000$ rows/s $\times$ $0.10/0.50/0.90$ prevalence factorial with a fixed one-second service deadline. Two additional protocols are recorded: *R3* was predeclared for controller-only power measurement but stopped before execution because its four-state design condition became ambiguous once the frozen artifact was inspected; *R4* is a post-hoc TinyDT threshold diagnostic conducted on validation only. R3 and R4 are reported in full because omitting either would misrepresent the executed evaluation surface.

VII. Static and Attribution Results

This section reports the classification operating points, the static physical resource characterization, and the primary/P1A attribution and robustness evidence. Together these anchor the pairwise non-degeneracy analysis of Section V in measured, rather than declared, cost profiles.

A. TON-IoT Detection Operating Points

Table I shows the four TON-IoT operating points used by the physical study. TinyDT has recall 0.9976 but F1 0.4792 and FPR 67.78%; LightLR, MedRF, and HeavyMLP have F1 0.9620–0.9746 and FPR 1.35–1.52%. MedRF's aggregate held-out balanced-accuracy advantage over LightLR is 0.0140; this post-freeze test statistic is descriptive and is not treated as a statewise attainability bound. The non-degeneracy conclusion instead uses the validation-only window audit in Section V, where the observed difference range $[-0.1053, 0.1053]$ does not straddle $\tau_{\text{MedRF,LightLR}} = 0.7967$. Full confusion counts, plus CICIoT2023/N-BaIoT detector and overlap audits, are retained in the supplement.

For attack prevalence p, recall r, and FPR f,

$$F1 = \frac{2pr}{p(1+r) + (1-p)f}, \quad (13)$$

so high attack prevalence can coexist with high F1 and poor normal-class behavior. This identity motivates our decision

TABLE II
P1A PAIRED CONTRASTS, TEN TRACE-MATCHED BLOCKS. ΔE IS FIRST-MINUS-SECOND METHOD; THE SIGN-FLIP p COLUMN IS AN EXACT NON-PARAMETRIC DIAGNOSTIC OVER THE TEN BLOCK-LEVEL DIFFERENCES.

Contrast	Outcome	Mean	95% CI
Tabular-Q − Static-LightLR	Energy (J)	$+0.456$	$[-1.861, +2.772]$
Tabular-Q − Static-LightLR	Power (mW)	$+0.910$	$[-3.722, +5.542]$
DQN − Static-LightLR	Energy (J)	$+4.525$	$[+2.693, +6.357]$
DQN − Static-LightLR	Power (mW)	$+9.045$	$[+5.381, +12.709]$
Static-MedRF − Static-LightLR	Energy (J)	$+12.471$	$[+10.611, +14.331]$
Static-MedRF − Static-LightLR	F1 (pp)	$+1.343$	$[+1.292, +1.393]$

to report attack-class F1, recall, and FPR jointly throughout, rather than summarising with F1 alone.

B. Static Physical Resource Measurements

Across fifteen static segments, idle averages 2.335 W and active means span 2.347 W (LightLR) to 2.397 W (MedRF). MedRF's paired increment is 61.6 mW with 95% CI $[32.1,\ 91.0]$; TinyDT and LightLR intervals cross zero, indicating that the whole-device signal at this cadence is dominated by background platform power. No accepted workload window records undervoltage or throttling. Full segment statistics are supplementary; labels such as "Tiny" or "Heavy" are not used as empirical cost ranks.

C. Primary Dynamic Comparison

All forty primary runs contain 501 power samples and 500 decisions. Relative to Static-MedRF, Tabular-Q saves 13.068 J (95% paired CI 9.736–16.400) and DQN saves 6.227 J (1.600–10.853), but both lose 1.343 F1 pp and 2.954 recall pp. Both select LightLR in all 5,000 windows; consequently the primary contrast measures a detector operating-point change and not adaptive switching. This is the core observation that motivates the P1A attribution control below.

D. P1A: Matched Attribution Against Static-LightLR

P1A executes ten matched blocks comparing Tabular-Q, DQN, Static-LightLR, and Static-MedRF on the same traces. Because both learned policies realize LightLR throughout, Static-LightLR is the correct baseline for isolating adaptive value. Table II reports the four key paired contrasts.

Three findings emerge. First, Tabular-Q's energy contrast against the matched Static-LightLR control has an interval that includes zero, so the primary campaign's 13.1-J saving against Static-MedRF is fully explained by detector operating-point choice rather than by adaptation. Second, DQN carries a small but resolvable +4.5-J energy premium over Static-LightLR despite reproducing identical detection, indicating that its per-window controller cost, while a tiny fraction of the second, is not free at 100 rows/s. Third, Static-MedRF's +12.5-J energy penalty against Static-LightLR is closely matched by the primary Tabular-Q saving against Static-MedRF, which confirms the compositional picture: (Static-MedRF − Static-LightLR) ≈ (Static-MedRF − Tabular-Q).

E. P1B: Isolated Scheduler Overhead

The isolated benchmark reports state-to-action decision time on the observed state support after correction of an earlier benchmarking substitution that was retained as an invalid attempt in the archive. On observed support, means are 0.739 μs for Static-LightLR, 6.582 μs for CFSM, 10.796 μs for Tabular-Q, and 1.567 ms for DQN. DQN is roughly $145\times$ Tabular-Q's controller cost but remains 0.157% of the one-second decision interval. These timings are not converted into joules; the R2 physical evidence in Section VIII bounds what can be attributed to controller cost alone.

The full 240-unit benchmark, confidence intervals, and raw timing arrays are retained in Supplementary Table S15.

F. P1C–P1E: Multi-seed and Sensitivity Coverage

P1C trains ten additional Tabular-Q and ten additional DQN policies under the frozen state, reward, split, and training-horizon protocol. All twenty policies select LightLR in 100% of held-out windows (Wilson 95% interval $[0.7225, 1.0]$). Greedy full-state maps vary across seeds—for DQN especially, from 0/324 to 311/324 TinyDT cells—but the observed-support behavior is unanimous, evidence for policy stability that is scoped to that support rather than to the whole encoded grid.

P1D varies the reward weights and separately ablates each state component; all 85 valid policies are single-action on held-out support. Only the five *detection-only* Tabular-Q variants—those with all resource penalties removed—switch the collapsed action away from LightLR to MedRF, describing these frozen runs rather than proving that resource-aware policies must choose LightLR. All thirty state-ablation policies remain single-action LightLR, indicating that no single state component was the mechanism suppressing switching. P1E adds 35 Tabular-Q runs across $\gamma \in \{0, 0.5, 0.9, 0.99\}$ and $\alpha \in \{0.05, 0.10, 0.20\}$; all thirty-five collapse to LightLR.

The full P1C–P1E matrices are retained in Supplementary Tables S16–S18. Together with Table II, they document the observed collapse through matched attribution, isolated timing, and sanctioned reward/state/hyperparameter perturbations. Only removing all resource penalties changes the collapsed detector in this matrix; this is not a proof of global optimality.

VIII. Dynamic Scheduler Results Under Fixed and Variable Load

This section reports the two prospectively frozen reviewer campaigns and the two diagnostic protocols. R1 stresses a fixed-load confidence cascade; R2 stresses a load- and prevalence-varying factorial with an explicit one-second deadline SLO. R3 is a predeclared stop and R4 is a validation-only calibration diagnostic. Together they measure when conditional selection adds value and when it does not.

A. R1: Fixed-Rate Confidence Cascade

Table III reports ten new matched blocks at 100 rows/s. The fixed cascade escalates 1.34% of rows and raises F1 by 1.347 pp over Static-LightLR while spending +7.952 J with

95% CI $[5.897, 10.007]$ J. Its raw mean-power difference is +15.917 mW $[11.798, 20.035]$, whereas the preceding-idle adjustment gives +13.635 mW $[-1.515, 28.784]$: environmental drift therefore weakens the power attribution but not the raw-energy result. Cascade F1 (0.98391) matches Static-MedRF (0.98386) within a few 10^{-5}, at 0.279 pp lower recall and 0.156 pp lower FPR.

Cascade cost is a call-granularity phenomenon, not a bound: The apparently disproportionate cascade latency at a 1.34% escalation rate is explained by call granularity. The frozen source performs *one* MedRF call per escalation-active window on the deferred sub-batch, not one MedRF call per deferred row. Although only 1.3388% of the 500,000 rows are deferred, they land in 3,338 of 5,000 windows (66.76%), with a mean of only 2.01 deferred rows per active window. No-escalation windows average 1.636 ms; any-escalation windows average 31.884 ms, close to the cost of a single MedRF batch call regardless of sub-batch size; the overall mean is 21.830 ms. This is an operating-point description, not a general bound: an optimized batched cascade with wider windows or amortized MedRF calls would show a different profile. What the measurement establishes is that reporting only the escalation *fraction* would be misleading; escalation *prevalence across windows* is the operational cost driver.

B. R2: Variable-Load Factorial and DeadlineGuard

R2 is the campaign in which conditional selection can, in principle, demonstrate adaptive value: every 500-second run visits each of the nine load-by-prevalence cells exactly once, followed by a 50-second recovery cell. All controllers see causal telemetry only; no method sees the factorial cell identity. Table IV reports the six-method means; Table V lists the two paired contrasts we treat as headline endpoints.

DeadlineGuard: causal switching adds measurable value: DeadlineGuard executes MedRF in 3,500 and LightLR in 1,500 of its 5,000 windows and records a mean 5.3 switches per run. Against Static-MedRF, it uses 43.714 J less per run (95% paired CI 41.418–46.010 J), a 2.98% reduction relative to the 1,468.1-J Static-MedRF mean, while cutting the deadline-miss rate by 10.24 pp to zero. The trade-off is 1.373 pp lower F1 and 2.853 pp lower recall. All ten energy differences share sign (exact sign-flip $p = 0.00195$). This is the campaign's positive result: a simple causal rule with a single binary observable achieves measurable energy savings and eliminates the SLO violations under variable load, an outcome that neither learned selector reproduces.

Against Static-LightLR, DeadlineGuard trades +7.550 J $[5.033, 10.068]$ for a +0.06-pp F1 gain and preserves zero deadline misses. This second contrast is deliberately less dramatic: Static-LightLR is already feasible under the deadline SLO at 4,000 rows/s because its inference is fast; DeadlineGuard buys back a small detection-quality gap by escalating to MedRF only where that gap matters. The comparison of the two contrasts is the design lesson: DeadlineGuard is not universally optimal; it is optimal against the high-security static baseline that becomes infeasible at the deadline, and moderate against the low-security static baseline that remains feasible.

The learned selectors fail to adapt under variable load: Frozen DQN remains LightLR in all 5,000 windows and reproduces the LightLR detection endpoints. Frozen Tabular-Q, in contrast, reaches new load-related states never visited during training; the zero-initialized Q rows are tied, `np.argmax` returns the first maximum, and TinyDT is action index zero. The registry labels this "fallback" but the frozen source contains no separate exception branch: the R2 outcome is the hardware realization of the off-support tie behavior already visible in the 314/324-state TinyDT greedy map of the primary Q table. The resulting run-level F1 is 0.7434 and FPR is 67.76%. This is a directly transferable safety risk of sparse discrete controllers whose default action ordering is not itself audited.

Heterogeneous variance and preceding-idle drift: R2 raw energy is heterogeneous across methods. For DQN versus Static-LightLR, all ten raw block differences are positive (median +5.72 J; exact sign-flip $p = 0.00195$), but a single Block-4 excursion is +392.0 J. That block's preceding idle is 3.919 W with a 49.2 °C maximum temperature, unlike the other DQN blocks. The excursion inflates the SD of paired differences to 122.2 J and expands the Student-t interval to $[-43.2, 131.7]$ J. Preceding-idle adjustment does not resolve the DQN–LightLR contrast. We therefore retain both the consistent raw direction and the unstable magnitude and *do not* claim a controller-power attribution, because R3 was not executed. This is exactly the sort of finding for which our protocol requires raw and drift-adjusted endpoints to be reported side by side. Figure 2 combines the corresponding occupancy, deadline, support, and security diagnostics.

C. R3: Protocol-Preserving Stop

R3 predeclared controller-only physical power over four "original observed-support states" for isolated attribution. The frozen artifact exposes eight such states and supplies no outcome-independent rule for selecting four. Choosing four *after the fact* would allow controller-power reporting to be driven by known R2 results, so the campaign stopped before measurement per the predeclared stop rule. No controller-only joule value is claimed. Reporting the stop rather than filling it silently preserves the integrity of the pairwise DQN–LightLR contrast above, which is why the DQN R2 magnitude is deliberately left as unresolved.

D. R4: Threshold Calibration Diagnostic

R4 selects a TinyDT threshold on validation by maximum balanced accuracy (highest-threshold tie-break), hashes it, and applies it *once* to test. Table VI contrasts the frozen primary threshold with the validation-selected diagnostic.

The diagnostic yields F1 0.9698, FPR 1.82%, recall 99.61%, BA 0.9889, and AUROC 0.9906. Recall changes by only 0.15 pp, so the primary failure at threshold 0.5 is substantially calibration-sensitive rather than a symptom of missing ranking information. Two consequences follow. First, TinyDT is not rehabilitated for the frozen physical campaigns: every classifier and scheduler artifact in those campaigns was fixed at threshold 0.5, and no post-hoc threshold can retroactively undo the unsafe TinyDT actions that Tabular-Q emitted under

TABLE III
R1 FIXED-RATE PHYSICAL RESULTS; MEANS OVER TEN PAIRED BLOCKS. ADJUSTED POWER SUBTRACTS THE IMMEDIATELY PRECEDING IDLE. ESCALATION IS THE FRACTION SENT FROM LIGHTLR TO MEDRF.

Method	Energy (J)	Power (W)	Adjusted (mW)	F1	Recall (%)	FPR (%)	Escalation (%)
Static-LightLR	1360.4	2.721	37.0	0.9704	96.71	1.36	–
Cascade	1368.3	2.737	50.6	0.9839	99.39	1.38	1.34
Tabular-Q	1362.0	2.724	39.7	0.9704	96.71	1.36	–
Static-MedRF	1368.6	2.737	56.5	0.9839	99.66	1.54	–

TABLE IV
R2 VARIABLE-LOAD RESULTS OVER TEN PAIRED BLOCKS. ENERGY IS MEAN±SD ACROSS RUNS; DEADLINE MISS IS THE FRACTION OF ONE-SECOND WINDOWS MISSING THE SERVICE DEADLINE; COMPLETION P95 IS THE 95TH-PERCENTILE WINDOW COMPLETION LATENCY.

Method	Energy (J)	Power (W)	F1	Recall (%)	FPR (%)	Miss (%)	Completion p95 (s)
Static-LightLR	1416.8±3.7	2.834	0.9763	96.68	1.35	0.00	0.070
Cascade	1448.1±46.4	2.896	0.9902	99.41	1.36	0.42	0.665
DeadlineGuard	1424.4±5.2	2.849	0.9769	96.80	1.36	0.00	0.069
Tabular-Q	1400.9±48.0	2.802	0.7434	99.76	67.76	0.00	0.063
DQN	1461.1±123.9	2.922	0.9763	96.68	1.35	0.00	0.072
Static-MedRF	1468.1±4.8	2.936	0.9906	99.65	1.52	10.24	1.013

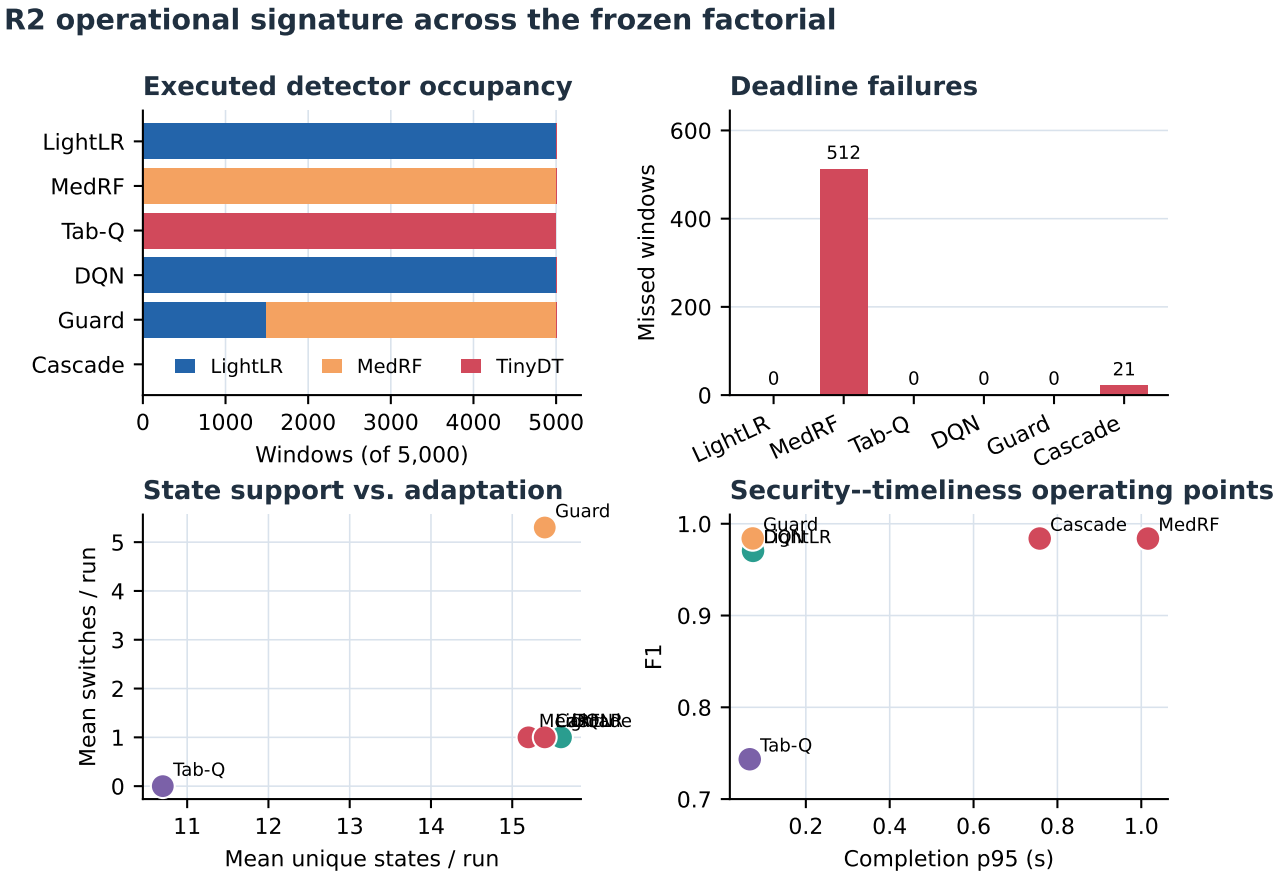


Fig. 2. R2 operational dashboard: realized detector occupancy, deadline misses, state support and switching, and security–timeliness operating points across the frozen 60-run factorial. DeadlineGuard is the single method with true load-conditioned switching and zero deadline misses; Tabular-Q's TinyDT collapse under new load states is visible in the occupancy panel.

R2. Second, calibration governance—validation-only threshold selection with a hashed freeze before test—is a deployable practice that we recommend for any operational IDS pipeline that includes a TinyDT-class detector.

IX. DISCUSSION

A. Attribution and Conditional Value

The primary 13.1-J Tabular-Q saving against Static-MedRF is only 26.1 mW over 500 s, while missed-positive presentations rise almost tenfold. P1A resolves no Tabular-Q difference

TABLE V
HEADLINE R2 PAIRED CONTRASTS, $n = 10$ BLOCKS. ΔE IS FIRST-MINUS-SECOND METHOD; DETECTION AND DEADLINE CHANGES ARE PERCENTAGE POINTS.

Contrast	ΔE (J), 95% CI	ΔF1	ΔMiss	sign-flip p
DeadlineGuard − Static-MedRF	-43.7 [-46.0,-41.4]	-1.373	-10.24	0.00195
DQN − Static-LightLR	+44.2 [-43.2,+131.7]	+0.000	+0.00	0.00195

TABLE VI
TON-IOT TINYDT TEST: FIXED PRIMARY VERSUS POST-HOC VALIDATION-SELECTED THRESHOLD. THE DIAGNOSTIC DOES NOT REPLACE THE PRIMARY THRESHOLD; IT ISOLATES CALIBRATION SENSITIVITY AS AN EXPLANATION FOR THE PRIMARY FAILURE.

Result	τ	TN	FP	FN	TP	F1	Rec.	FPR	BA
Primary fixed	0.500	5519	11610	13	5348	0.4792	99.76	67.78	0.6599
Validation-selected diagnostic	0.646751	16817	312	21	5340	0.9698	99.61	1.82	0.9889

against the matched Static-LightLR action, so this is detector choice rather than adaptation. R2 supplies the positive contrast: DeadlineGuard uses the causal arrival count to switch, removes Static-MedRF's 512 deadline misses, and saves 43.7 J at a 1.373-pp F1 cost. The confidence cascade reaches MedRF-like F1, but its 1.34% deferred rows activate MedRF in 66.76% of windows; call granularity, not row fraction alone, controls serving cost.

B. External-Budget Normalization Rule

Theorem 1 makes reward design a deployment contract. Candidate-range scaling yields $\tau_{\text{MedRF,LightLR}} = 0.7967$, outside the observed validation-window difference range $[-0.1053, 0.1053]$, and therefore suppresses reversal on that support. A portable design replaces pool extrema with external denominators:

$$\bar{P}_a = \frac{P_a - P_{\text{idle}}}{B_P}, \qquad \bar{L}_a = \frac{L_a}{\Delta_{\text{SLO}}}, \tag{14}$$

$$\bar{E}_a = \frac{E_a}{B_E}, \qquad \bar{H}_a = \frac{[T_a - T_{\text{safe}}]_+}{B_T}. \tag{15}$$

where B_P, Δ_{SLO}, B_E, and B_T are operator-set power, latency, energy, and thermal budgets. These denominators remain interpretable when the detector pool changes. The validation-only audit should be run before policy freeze; if no relevant pair can reverse, use an audited static schedule or redesign the pre-test contract.

C. Support, Safety, and Measurement

Frozen Tabular-Q maps 314/324 states to TinyDT because unseen zero-initialized rows tie and `np.argmax` selects action zero. R2 enters those new load states and raises FPR to 67.76%; DQN remains LightLR, but seed-dependent global maps show that this is not validated off-support generalization. Deployments should therefore archive state coverage, full action maps, and the security quality of tied/default actions.

Small whole-device effects also require paired blocks and joint raw/drift-adjusted reporting. In R2, DQN-minus-LightLR is positive in all ten raw blocks, yet one thermal/power excursion widens the interval across zero; neither that direction nor the adjustment supports a controller-power claim because R3 stopped before measurement. The practical checklist is concise: audit reward attainability; include the realized static action and simple causal controls; freeze a load/threat stress factorial; report confusion counts, deadlines, occupancy, and off-support defaults; retain both raw and adjusted power endpoints; and select detector thresholds on validation only.

The portable contribution is this audit-and-attribution framework, not a universal numeric ranking. The 43.7-J DeadlineGuard saving, 66.76% cascade activation, and controller timings are specific to the Pi 4B, resident pool, windowing regime, and meter chain used here.

X. LIMITATIONS AND REPRODUCIBILITY BOUNDARY

Classifier results use one training seed, fixed dataset-native splits, and threshold 0.5; replay reuses held-out pools. Only TON-IoT receives dynamic physical testing. Ten paired blocks on one Raspberry Pi 4B 8 GB do not estimate board, meter, site, or fleet variability, and CICIoT2023/N-BaIoT are not dynamic-power evidence.

The ChargerLAB POWER-Z KM003C (HW 2.4, FW 2.0.6) was not independently laboratory calibrated. Its nominal 1-Hz load-side USB measurements exclude AC conversion, packet capture, and feature encoding. Models remain resident, temperatures stay low, and the factorial is scheduled feature replay rather than live traffic. The software-policy traces contain only a constant 35°C predecision fallback and no action-attributable post-action temperature; the executed thermal reward is therefore zero and cannot establish thermal-aware adaptation. Pairing cannot remove ambient drift; R3 supplies no controller-only joule value; and R1 cascade cost is specific to 100-row windows.

P1 controls and R4 are post-hoc to the primary campaign; R1/R2 are prospectively frozen only for their own execution. The reward audit is retrospective, and cannot establish execution before test inspection. Its earlier implementation read aggregate test confusion counts for a descriptive diagnostic; the corrected entry point excludes those inputs. The corrected software adapter reports missing post-action temperature as null and flags the unavailable measurement; the thermal penalty is omitted rather than treated as a measured zero. Historical results are retained unchanged. A future temperature-increment experiment requires separate pre/post measurements on a common timing basis; a predecision EMA is a state feature, not a measured pre/post increment. No equivalence margin was declared. Invalid attempts and authorized replacements remain archived. Manifests, CRC, source hashes, and leakage validators establish integrity, not external validity. The theorem assumes a linear utility with $w_D > 0$ and support-fixed normalized costs; non-linear objectives, switching penalties, and load-dependent costs require support-conditioned extensions.

XI. CONCLUSION

The pairwise non-degeneracy criterion tests action-pair reversal on a specified support region, while matched-static attribution distinguishes adaptation from detector choice. In our frozen study, Tabular-Q's apparent primary saving is

explained by its LightLR operating point; off support its deterministic tie rule selects TinyDT and reaches 67.76% FPR. By contrast, DeadlineGuard uses a direct causal load signal to save 43.7 J against Static-MedRF (2.98%; 95% CI 41.4–46.0 J) and remove all 512 deadline misses at a 1.373-pp F1 cost. Learned orchestration should be credited only after reward attainability, realized actions, matched controls, service feasibility, off-support defaults, and whole-device measurement are audited together.

REFERENCES

[1] N. Wang *et al.*, "Intelligent driving task scheduling service in vehicle-edge collaborative networks based on deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4357–4368, 2024.

[2] F. W. Murti, S. Ali, G. Iosifidis, and M. Latva-aho, "Deep reinforcement learning for orchestrating cost-aware reconfigurations of vrans," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 200–216, 2024.

[3] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, "Dvfo: Learning-based dvfs for energy-efficient edge-cloud collaborative inference," *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9042–9059, 2024.

[4] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2389–2401, 2024.

[5] X. Zhang *et al.*, "Aoc-ids: Autonomous online framework with contrastive learning for intrusion detection," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024.

[6] P. S. Bouzinis, P. Radoglou-Grammatikis, I. Makris, T. Lagkas, V. Argyriou, G. T. Papadopoulos, P. Sarigiannidis, and G. K. Karagiannidis, "Statavg: Mitigating data heterogeneity in federated learning for intrusion detection systems," *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 2944–2955, 2025.

[7] C. Zha *et al.*, "Dm-ids—a network intrusion detection method based on dual-modal fusion," *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 3646–3661, 2025.

[8] Y.-D. Lin, H.-X. Huang, D. Sudyana, and Y.-C. Lai, "Ai for ai-based intrusion detection as a service: Reinforcement learning to configure models, tasks, and capacities," *Journal of Network and Computer Applications*, vol. 229, p. 103936, 2024.

[9] M. M. Hasan, R. Islam, Q. Mamun, M. Z. Islam, and J. Gao, "Ensemble-based adversarial defense with dynamic model selection for intrusion detection systems," *Internet of Things*, vol. 38, p. 101970, 2026.

[10] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.

[11] T. Bolukbasi, J. Wang, O. Dekel, and V. Saligrama, "Adaptive neural networks for efficient inference," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 527–536.

[12] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018.

[13] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 3301–3310.

[14] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, 2017, pp. 613–627.

[15] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 2021, pp. 397–411.

[16] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 443–462.

[17] J. Lou, Z. Tang, W. Jia, W. Zhao, and J. Li, "Startup-aware dependent task scheduling with bandwidth constraints in edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 2, pp. 1586–1600, 2024.

[18] M. Kim, J. Jang, Y. Choi, and H. J. Yang, "Distributed task offloading and resource allocation for latency minimization in mobile edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 15 149–15 166, 2024.

[19] E. F. Maleki, W. Ma, L. Mashayekhy, and H. J. L. Roche, "Qos-aware content delivery in 5g-enabled edge computing: Learning-based approaches," *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9324–9336, 2024.

[20] Z. Nan, Y. Han, S. Zhou, and Z. Niu, "Robust dnn partitioning and resource allocation under uncertain inference time," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3680–3696, 2026.

[21] J. Wang, Z. Li, H. Liu, T. Qiu, and H. Luo, "A trust-based computation offloading framework in mobile cloud-edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 6, pp. 5370–5385, 2025.

[22] Z. Hu, J. Niu, T. Ren, X. Liu, and M. Guizani, "Sitoff: Enabling size-insensitive task offloading in d2d-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 3, pp. 1567–1584, 2025.

[23] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[24] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[25] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[26] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," arXiv:2108.13264, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[27] UNSW Canberra, "ToN-IoT datasets," Institutional dataset description, accessed: Sep. 7, 2026. [Online]. Available: https://research.unsw.edu.au/projects/toniot-datasets

[28] Canadian Institute for Cybersecurity, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," University of New Brunswick, accessed: Sep. 7, 2026. [Online]. Available: https://www.unb.ca/cic/datasets/iotdataset-2023.html

[29] UCI Machine Learning Repository, "Detection of IoT Botnet Attacks N-BaIoT," Dataset 442, accessed: Sep. 7, 2026. [Online]. Available: https://archive.ics.uci.edu/dataset/442/detection+of+iot+botnet+attacks+n+baiot

[30] NIST/SEMATECH, "Confidence limits for the mean," e-Handbook of Statistical Methods, Sec. 1.3.5.2, accessed: Sep. 7, 2026. [Online]. Available: https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm